

JAVASCRIPT PROGRAMMING

Teaching Notes

MODULE 3: Functions and Arrays

Module Overview

This module introduces two of the most important building blocks in JavaScript: **functions** and **arrays**. Functions let you write reusable, organised code. Arrays let you store and manage collections of data. Together they unlock the ability to write powerful, efficient programs.

Duration: Week 4 | Suggested Time: 6–9 hours | Practical: at least 50%

Learning Outcomes

By the end of this module, learners should be able to:

- Define functions and explain why they are used
- Write functions with parameters and return values
- Understand scope: local vs global variables
- Create, access, and modify arrays
- Loop through arrays using `for` and `forEach`
- Use built-in array methods: `push`, `pop`, `shift`, `unshift`, `splice`, `slice`, `map`, `filter`, `find`, `includes`
- Use the `Math` object and its key methods

- Use array properties: length and join

LESSON 1: Introduction to Functions

1.1 What is a Function?

A function is a named, reusable block of code designed to perform a specific task. Instead of writing the same code over and over, you define it once in a function and call it whenever you need it.

Why Use Functions?

- **Reusability** — write once, use many times
- **Organisation** — break a large program into smaller, manageable pieces
- **Readability** — well-named functions make code self-documenting
- **Maintainability** — fix a bug in one place instead of many

1.2 Function Syntax and Structure

A function is defined using the **function** keyword, followed by a name, parentheses, and a code block in curly braces.

```
// Basic function syntax:

function functionName(parameter1, parameter2) {

  // code to execute

  return result; // optional

}

// Example – a function that greets a user:

function greetUser(name) {

  let message = "Hello, " + name + "! Welcome to JavaScript.";

  return message;

}
```

```
// Calling (invoking) the function:

let greeting = greetUser("Alice");

console.log(greeting);

// Output: Hello, Alice! Welcome to JavaScript.
```

1.3 Invoking (Calling) a Function

Defining a function does not run it. You must **call** (invoke) it by writing its name followed by parentheses.

```
function sayHello() {

  console.log("Hello, World!");

}

// The function is defined above but hasn't run yet.

// Calling it:

sayHello(); // Output: Hello, World!

sayHello(); // Can call it multiple times

sayHello(); // Output: Hello, World!
```

1.4 Parameters and Arguments

Parameters are the variable names listed in the function definition. Arguments are the actual values passed when the function is called.

```
// parameters

// ↓ ↓

function add(a, b) {

  return a + b;

}
```

```
// arguments

// ↓ ↓ ↓

let result = add(10, 5);

console.log(result); // 15

// Multiple calls with different arguments:

console.log(add(100, 250)); // 350

console.log(add(3.5, 1.5)); // 5
```

Default Parameters (ES6)

You can give parameters default values that are used when no argument is provided.

```
function greet(name = "Guest", language = "English") {

  if (language === "Swahili") {

    return "Habari, " + name + "!";

  }

  return "Hello, " + name + "!";

}

console.log(greet("Brian", "Swahili")); // Habari, Brian!

console.log(greet("Carol")); // Hello, Carol!

console.log(greet()); // Hello, Guest!
```

1.5 Return Values

The **return** statement sends a value back to wherever the function was called. Once return is reached, the function stops executing.

```
function calculateVAT(price) {
```

```
let vat = price * 0.16;

return vat; // sends the value back

}

let vatAmount = calculateVAT(5000);

console.log("VAT: KES " + vatAmount); // VAT: KES 800

// You can use the return value directly:

console.log("Total: KES " + (5000 + calculateVAT(5000))); // Total: KES 5800

// A function without return gives undefined:

function noReturn() {

  console.log("I do not return anything");

}

let x = noReturn();

console.log(x); // undefined
```

1.6 Scope: Local vs Global Variables

Scope refers to where a variable can be accessed in your code.

Scope	Description
Global scope	Variables declared outside any function. Accessible everywhere in the script.
Local scope	Variables declared inside a function. Only accessible within that function.
Block scope	Variables declared with let or const inside { } blocks. Only accessible within that block.

```
let country = "Kenya"; // global variable

function showLocation() {
```

```
let city = "Nairobi"; // local variable

console.log(country); // OK – can access global

console.log(city); // OK – local variable

}

showLocation();

console.log(country); // OK – global

// console.log(city); // ERROR – city is not defined here

// Block scope example:

if (true) {

  let blockVar = 'I am block-scoped';

  console.log(blockVar); // OK

}

// console.log(blockVar); // ERROR – outside the block
```

Best Practice

Keep variables as local as possible. Avoid creating too many global variables — they can be accidentally overwritten from anywhere in your code, causing hard-to-find bugs.

LESSON 2: Types of Functions

JavaScript supports several ways to define a function. Each type has its own syntax, behaviour, and best use case. Understanding all of them helps you read and write modern JavaScript confidently.

The 4 Types of Functions in JavaScript

Here is an overview of all the function types covered in this lesson:

- **1. Function Declaration** — the classic, named function defined with the `function` keyword. Hoisted — can be called before it appears in the code.
- **2. Function Expression** — a function stored inside a variable. Not hoisted — must be defined before calling.
- **3. Arrow Function** — a compact ES6 syntax using `=>`. Ideal for short functions and callbacks.
- **4. Anonymous Function** — a function with no name, used inline as an argument to another function.

Each type is explained in full with examples on the pages that follow.

2.1 Function Declaration

The standard way to define a function. These are **hoisted** — meaning they can be called before they are defined in the code.

```
// Call before definition – works due to hoisting:

console.log(square(4)); // 16

function square(num) {

  return num * num;

}
```


2.2 Function Expression

A function stored in a variable. These are **not hoisted** — they must be defined before being called.

```
const multiply = function(a, b) {  
  
  return a * b;  
  
};  
  
console.log(multiply(3, 7)); // 21  
  
// Cannot be called before the line above:  
  
// multiply(3, 7) // ERROR if placed before the definition
```

2.3 Arrow Functions (ES6)

A shorter syntax for writing function expressions. Arrow functions are very common in modern JavaScript.

```
// Regular function expression:  
  
const add = function(a, b) {  
  
  return a + b;  
  
};  
  
// Arrow function – equivalent:  
  
const add = (a, b) => {  
  
  return a + b;  
  
};  
  
// Even shorter – implicit return (single expression, no braces):  
  
const add = (a, b) => a + b;  
  
console.log(add(4, 6)); // 10
```

```
// Single parameter – parentheses optional:
```

```
const double = n => n * 2;

console.log(double(7)); // 14
```

```
// No parameters – empty parentheses required:
```

```
const greet = () => 'Hello!';

console.log(greet()); // Hello!
```

2.4 Anonymous Functions

Functions without a name, often used as arguments passed to other functions (callbacks).

```
// Anonymous function as an argument:
```

```
setTimeout(function() {

  console.log("This runs after 2 seconds");

}, 2000);
```

```
// Arrow function as callback (more common modern style):
```

```
setTimeout(() => {

  console.log("Arrow version – also runs after 2 seconds");

}, 2000);
```

2.5 Function Summary Table

Type	Syntax	Notes
Declaration	function name() {}	Hoisted — can call before definition
Expression	const f = function() {}	Not hoisted — define before calling
Arrow	const f = () => {}	ES6 — concise, common in callbacks

Anonymous	<code>function() {}</code>	No name — used inline as argument
-----------	----------------------------	-----------------------------------

LESSON 3: Arrays — Creating and Accessing

3.1 What is an Array?

An array is an ordered collection of values stored in a single variable. Each value is called an **element**, and each element has a numbered position called an **index**, starting at **0**.

Analogy

Think of an array as a numbered shelf. Shelf 0 holds the first item, shelf 1 the second, and so on. You can store anything on any shelf, and you always know exactly where each item is by its number.

3.2 Creating Arrays

```
// Array literal (most common way):

let fruits = ["mango", "banana", "orange", "avocado"];


// Mixed data types (allowed, but uncommon):

let mixed = [42, "hello", true, null];


// Empty array:

let emptyList = [];


// Array constructor (less common):

let numbers = new Array(1, 2, 3, 4, 5);


// Array of objects:

let students = [

  { name: "Alice", grade: "A" },

  { name: "Bob", grade: "B" },

  { name: "Carol", grade: "A" }
```

```
];
```

3.3 Accessing Elements

Use square bracket notation with the index to access any element. Remember: indexing starts at **0**.

```
let colours = ["red", "green", "blue", "yellow"];

// index: 0 1 2 3

console.log(colours[0]); // red

console.log(colours[2]); // blue

console.log(colours[3]); // yellow

// Accessing the LAST element:

console.log(colours[colours.length - 1]); // yellow

// Accessing an element that doesn't exist:

console.log(colours[10]); // undefined

// Accessing elements in an array of objects:

console.log(students[0].name); // "Alice"

console.log(students[1].grade); // "B"
```

3.4 Modifying Elements

Assign a new value using the index to update an element.

```
let cities = ["Nairobi", "Mombasa", "Kisumu", "Nakuru"];

// Update an element:

cities[2] = "Eldoret";
```

```
console.log(cities); // ["Nairobi", "Mombasa", "Eldoret", "Nakuru"]

// Add a new element at the end (by index):

cities[4] = "Thika";

console.log(cities); // [..., "Nakuru", "Thika"]
```

3.5 The length Property

The **length** property tells you how many elements are in an array.

```
let animals = ["lion", "elephant", "giraffe", "zebra", "cheetah"];

console.log(animals.length); // 5

// length is dynamic – updates automatically when you add/remove items:

animals.push("rhino");

console.log(animals.length); // 6

// Use length to safely access the last element:

let last = animals[animals.length - 1];

console.log(last); // "rhino"
```

LESSON 4: Looping Through Arrays

Looping lets you process every element in an array without writing repetitive code. JavaScript provides several ways to loop through arrays.

4.1 for Loop

The classic loop — gives full control over the index. Best when you need the index number or want to loop in a specific range.

```
let marks = [78, 85, 62, 91, 70];

for (let i = 0; i < marks.length; i++) {

  console.log(`Student ${i + 1}: ${marks[i]}%`);

}

// Student 1: 78%

// Student 2: 85%

// ... and so on


// Calculate the total using a for loop:

let total = 0;

for (let i = 0; i < marks.length; i++) {

  total += marks[i];

}

let average = total / marks.length;

console.log(`Class average: ${average}%`); // 77.2%
```

4.2 for...of Loop (ES6)

A cleaner way to loop through arrays when you only need the **value** (not the index).

```
let subjects = ["Maths", "English", "Science", "ICT", "Kiswahili"];

for (let subject of subjects) {

  console.log(`Subject: ${subject}`);

}

// Subject: Maths

// Subject: English

// ... and so on
```

4.3 forEach() Method

The `forEach()` method runs a function once for each element. It takes a **callback function** as an argument.

```
let products = ["Laptop", "Phone", "Tablet", "Headphones"];

// Using an arrow function callback:

products.forEach((product, index) => {

  console.log(`${index + 1}. ${product}`);

});

// 1. Laptop

// 2. Phone

// 3. Tablet

// 4. Headphones

// forEach with just the value:

let prices = [1500, 25000, 45000, 3500];

let grandTotal = 0;
```



```
prices.forEach(price => {  
  
  grandTotal += price;  
  
});  
  
console.log(`Grand Total: KES ${grandTotal}`); // KES 75000
```

4.4 Looping Summary

Method	Best for	Access to index?
for	Full control, counting, reverse loops	Yes
for...of	Simple value iteration (clean syntax)	No (use entries() if needed)
forEach()	Performing an action on each element	Yes (2nd callback param)

LESSON 5: Array Methods

JavaScript arrays come with a rich set of built-in methods. These are functions you can call directly on any array to add, remove, search, transform, and filter data.

5.1 Adding and Removing Elements

Method	Action	Returns
push(item)	Add item(s) to the END	New length
pop()	Remove item from the END	Removed item
unshift(item)	Add item(s) to the BEGINNING	New length
shift()	Remove item from the BEGINNING	Removed item
splice(i,n,...)	Remove/replace/insert at position	Removed items array

```
let queue = ["Alice", "Bob", "Carol"];

// push – add to end:

queue.push("David");

console.log(queue); // ["Alice", "Bob", "Carol", "David"]

// pop – remove from end:

let last = queue.pop();

console.log(last); // "David"

console.log(queue); // ["Alice", "Bob", "Carol"]

// unshift – add to beginning:

queue.unshift("Zara");
```

```
console.log(queue); // ["Zara", "Alice", "Bob", "Carol"]

// shift – remove from beginning:

let first = queue.shift();

console.log(first); // "Zara"

console.log(queue); // ["Alice", "Bob", "Carol"]

// splice(startIndex, deleteCount, ...itemsToInsert):

queue.splice(1, 1, "Eve", "Frank"); // at index 1, remove 1, insert 2

console.log(queue); // ["Alice", "Eve", "Frank", "Carol"]
```

5.2 Extracting Parts — slice()

The `slice(start, end)` method returns a **new array** with elements from start up to (but not including) end. The original array is unchanged.

```
let letters = ["a", "b", "c", "d", "e", "f"];

let part = letters.slice(1, 4);

console.log(part); // ["b", "c", "d"]

console.log(letters); // ["a", "b", "c", "d", "e", "f"] – unchanged

// slice from index 3 to end:

console.log(letters.slice(3)); // ["d", "e", "f"]

// Negative index – count from the end:

console.log(letters.slice(-2)); // ["e", "f"] (last 2 elements)
```

5.3 Searching — indexOf(), includes(), find()

```
let colours = ["red", "green", "blue", "yellow", "green"];

// indexOf – returns first index of a value (-1 if not found):

console.log(colours.indexOf("green")); // 1

console.log(colours.indexOf("purple")); // -1

// includes – returns true/false:

console.log(colours.includes("blue")); // true

console.log(colours.includes("white")); // false

// find – returns the FIRST element that matches a condition:

let scores = [45, 72, 38, 91, 60];

let firstPass = scores.find(score => score >= 50);

console.log(firstPass); // 72

// findIndex – returns the INDEX of the first match:

let firstPassIndex = scores.findIndex(score => score >= 50);

console.log(firstPassIndex); // 1
```

5.4 Transforming — map()

`map()` creates a **new array** by applying a function to every element. The original array is not modified.

```
let prices = [100, 250, 450, 1200];

// Add 16% VAT to every price:

let pricesWithVAT = prices.map(price => price * 1.16);

console.log(pricesWithVAT); // [116, 290, 522, 1392]
```

```
// Convert names to uppercase:

let names = ["alice", "bob", "carol"];

let upperNames = names.map(name => name.toUpperCase());

console.log(upperNames); // ["ALICE", "BOB", "CAROL"]


// Extract a single property from an array of objects:

let students = [

  { name: "Grace", score: 88 },

  { name: "James", score: 74 },

  { name: "Mary", score: 91 }

];

let studentNames = students.map(s => s.name);

console.log(studentNames); // ["Grace", "James", "Mary"]
```

5.5 Filtering — filter()

filter() creates a **new array** containing only elements that pass a test (return true from the callback).

```
let scores = [45, 72, 38, 91, 60, 55, 29];

// Keep only passing scores (>= 50):

let passing = scores.filter(score => score >= 50);

console.log(passing); // [72, 91, 60, 55]


// Filter objects:

let products = [

  { name: "Laptop", price: 85000, inStock: true },
```

```
{ name: "Phone", price: 45000, inStock: false },  
  
{ name: "Tablet", price: 32000, inStock: true },  
  
{ name: "Watch", price: 12000, inStock: false },  
  
];  
  
let available = products.filter(p => p.inStock);  
  
console.log(available.map(p => p.name));  
  
// ["Laptop", "Tablet"]  
  
let affordable = products.filter(p => p.price <= 45000);  
  
console.log(affordable.length); // 3
```

5.6 Joining — join()

`join(separator)` combines all array elements into a single string, separated by the specified separator.

```
let words = ["JavaScript", "is", "fun", "to", "learn"];  
  
console.log(words.join(" ")); // "JavaScript is fun to learn"  
  
console.log(words.join("-")); // "JavaScript-is-fun-to-learn"  
  
console.log(words.join(", ")); // "JavaScript, is, fun, to, learn"  
  
console.log(words.join("")); // "JavaScriptisfuntolearn"  
  
// Useful for building CSV-like strings:  
  
let headers = ["Name", "Age", "Course", "Grade"];  
  
console.log(headers.join(",")); // Name,Age,Course,Grade
```

5.7 Other Useful Array Methods

Method	What it does
concat(arr2)	Merges two arrays into a new array
reverse()	Reverses the array in place
sort()	Sorts alphabetically (use a comparator for numbers)
flat()	Flattens nested arrays one level
reduce(fn, initial)	Reduces array to a single value (e.g. sum)
every(fn)	Returns true if ALL elements pass the test
some(fn)	Returns true if AT LEAST ONE element passes the test
fill(value, start, end)	Fills array with a static value
Array.isArray(x)	Returns true if x is an array

```
// reduce – summing all values:
```

```
let nums = [10, 20, 30, 40, 50];
```

```
let sum = nums.reduce((accumulator, current) => accumulator + current, 0);
```

```
console.log(sum); // 150
```

```
// sort numbers correctly (default sort is alphabetical!):
```

```
let prices = [200, 50, 1000, 75, 500];
```

```
prices.sort((a, b) => a - b); // ascending
```

```
console.log(prices); // [50, 75, 200, 500, 1000]
```

```
// every and some:
```

```
let ages = [20, 25, 18, 30, 22];
```

```
console.log(ages.every(a => a >= 18)); // true – all adults
```

```
console.log(ages.some(a => a >= 25)); // true – at least one
```

LESSON 6: The Math Object

The **Math** object is a built-in JavaScript object that provides mathematical constants and functions. You do not need to create it — it is always available.

6.1 Math Properties (Constants)

Property	Value and Description
Math.PI	3.14159... — ratio of circumference to diameter of a circle
Math.E	2.71828... — Euler's number (base of natural logarithm)
Math.SQRT2	1.41421... — square root of 2
Math.LN2	0.69314... — natural logarithm of 2
Math.LN10	2.30258... — natural logarithm of 10

6.2 Math Methods

Method	Description and Example
Math.round(x)	Round to nearest integer: <code>Math.round(4.6) → 5</code>
Math.floor(x)	Round DOWN: <code>Math.floor(4.9) → 4</code>
Math.ceil(x)	Round UP: <code>Math.ceil(4.1) → 5</code>
Math.abs(x)	Absolute value: <code>Math.abs(-7) → 7</code>
Math.sqrt(x)	Square root: <code>Math.sqrt(25) → 5</code>
Math.pow(base, exp)	Power: <code>Math.pow(2, 10) → 1024</code>
Math.max(...nums)	Largest value: <code>Math.max(3, 1, 9, 2) → 9</code>
Math.min(...nums)	Smallest value: <code>Math.min(3, 1, 9, 2) → 1</code>
Math.random()	Random decimal between 0 (inclusive) and 1 (exclusive)
Math.trunc(x)	Remove decimal part: <code>Math.trunc(4.9) → 4</code>
Math.log(x)	Natural logarithm of x

```
// Practical examples:
```

```
let radius = 7;
```



```
let area = Math.PI * Math.pow(radius, 2);

console.log(`Circle area: ${area.toFixed(2)}`); // 153.94

// Random integer between min and max (inclusive):

function randomInt(min, max) {

  return Math.floor(Math.random() * (max - min + 1)) + min;

}

console.log(randomInt(1, 6)); // random dice roll

console.log(randomInt(1, 100)); // random percentage

// Find the highest and lowest scores:

let scores = [78, 92, 55, 88, 63, 97, 41];

console.log(`Highest: ${Math.max(...scores)}`); // 97

console.log(`Lowest: ${Math.min(...scores)}`); // 41
```

PRACTICAL SESSION: Functions + Arrays

Practical Objective

Learners will write programs that combine functions and arrays — building reusable logic and working with collections of data. All exercises should be tested in the browser console or a code editor.

Exercise 1: Reusable Grade Calculator

Write a function that takes a score and returns the grade. Then use it with an array of scores.

```
function getGrade(score) {  
  
  if (score >= 70) return 'Distinction';  
  
  if (score >= 60) return 'Credit';  
  
  if (score >= 50) return 'Pass';  
  
  return 'Fail';  
  
}  
  
let classScores = [88, 45, 72, 61, 55, 39, 94, 68];  
  
// Print each score and its grade:  
  
classScores.forEach((score, index) => {  
  
  console.log(`Student ${index + 1}: ${score}% — ${getGrade(score)}`);  
  
});  
  
// Count how many passed:  
  
let passed = classScores.filter(s => s >= 50);  
  
console.log(`Passed: ${passed.length} out of ${classScores.length}`);  
  
// Calculate class average:
```

```
let total = classScores.reduce((sum, s) => sum + s, 0);

let average = (total / classScores.length).toFixed(1);

console.log(`Class average: ${average}%`);
```

Exercise 2: Shopping Cart

Build a simple shopping cart using an array of objects and functions.

```
let cart = [];

function addItem(name, price, qty) {

  cart.push({ name, price, qty });

  console.log(`Added: ${qty}x ${name}`);

}

function getSubtotal(item) {

  return item.price * item.qty;

}

function printReceipt() {

  console.log('==== RECEIPT =====');

  let total = 0;

  cart.forEach(item => {

    let subtotal = getSubtotal(item);

    total += subtotal;

    console.log(`${item.name} x${item.qty} = KES ${subtotal}`);

  });

}
```

```
let vat = total * 0.16;

console.log(`-----`);

console.log(`Subtotal: KES ${total}`);

console.log(`VAT 16%: KES ${vat.toFixed(2)}`);

console.log(`TOTAL: KES ${total + vat}.toFixed(2)}`);

}

addItem('Bread', 60, 2);

addItem('Milk', 55, 3);

addItem('Eggs', 180, 1);

printReceipt();
```

Exercise 3: Math Utilities

Write a set of utility functions using the Math object.

```
// Function to generate a random integer:

const randInt = (min, max) => Math.floor(Math.random() * (max - min + 1)) + min;

// Function to calculate circle area:

const circleArea = r => +(Math.PI * r * r).toFixed(2);

// Function to find stats for an array of numbers:

function getStats(nums) {

  return {

    count: nums.length,

    sum: nums.reduce((a, b) => a + b, 0),
```

```
average: +(nums.reduce((a, b) => a + b, 0) / nums.length).toFixed(2),

highest: Math.max(...nums),

lowest: Math.min(...nums),

};

}

// Test with exam scores:

let scores = [78, 92, 55, 88, 63, 97, 41, 76];

console.log(getStats(scores));

// { count: 8, sum: 590, average: 73.75, highest: 97, lowest: 41 }

// Generate 5 random dice rolls:

let rolls = Array.from({ length: 5 }, () => randInt(1, 6));

console.log('Dice rolls:', rolls);
```

Challenge Exercise

Challenge: Student Report System

Build a program that:

- Stores an array of at least 6 student objects, each with: name, scores (array of 3 unit scores)
- Write a function `getAverage(scores)` that returns the average of an array of scores
- Write a function `getGrade(avg)` that returns Distinction / Credit / Pass / Fail
- Use `map()` to create a results array where each student has their average and grade added
- Use `filter()` to find all students who got a Distinction

- Use `sort()` to rank students from highest to lowest average
- Print a formatted report to the console

Bonus: Find the overall class average and the student with the highest score using `Math.max()`.

MODULE 3 SUMMARY

Key Takeaways

1. **Functions** are reusable blocks of code. Define them with `function`, then call them by name with parentheses.
2. **Parameters** are placeholders in the definition; **arguments** are the actual values passed when calling.
3. **return** sends a value back from a function. Without it, the function returns `undefined`.
4. **Scope** — variables declared inside a function are local and not accessible outside it.
5. **Arrow functions** `() => {}` are a concise ES6 syntax for writing functions.
6. **Arrays** store ordered collections. Access elements with `array[index]`, starting at index 0.
7. **Array methods**: `push/pop/shift/unshift` modify the array; `map/filter` create new arrays; `forEach` iterates; `reduce` collapses to one value.
8. **The Math object** provides constants like `Math.PI` and methods like `Math.round()`, `Math.random()`, `Math.max()`.

Quick Reference — Array Methods

Method	Modifies original?	Returns
<code>push()</code> / <code>pop()</code>	Yes	New length / removed item
<code>shift()</code> / <code>unshift()</code>	Yes	Removed item / new length
<code>splice()</code>	Yes	Array of removed items
<code>slice()</code>	No	New sub-array
<code>map()</code>	No	New transformed array
<code>filter()</code>	No	New filtered array
<code>forEach()</code>	No	undefined

<code>find()</code>	No	First matching element
<code>includes()</code>	No	true or false
<code>indexOf()</code>	No	Index or -1
<code>join()</code>	No	String
<code>sort()</code>	Yes	Sorted array
<code>reverse()</code>	Yes	Reversed array
<code>reduce()</code>	No	Single accumulated value

Review Questions

1. What is a function and why do we use them? Give two benefits.
2. What is the difference between a **parameter** and an **argument**? Give an example.
3. What does the `return` statement do? What happens if a function has no return statement?
4. Explain the difference between **global scope** and **local scope**.
5. Rewrite this function as an arrow function: `function double(n) { return n * 2; }`
6. Given `let arr = [10, 20, 30, 40, 50]`, write code to: (a) add 60 to the end, (b) remove the first element, (c) get the last element.
7. What is the difference between `splice()` and `slice()`?
8. What does `map()` do differently from `forEach()`?
9. Write a one-line expression using `filter()` to get all numbers greater than 50 from: `[30, 70, 45, 88, 12, 55]`
10. What does `Math.random()` return? How would you use it to generate a random number between 1 and 10?

Looking Ahead — Module 4

*In Module 4 we explore the **Document Object Model (DOM)** — how JavaScript reads and manipulates the actual HTML elements on a web page. This is where JavaScript becomes truly interactive, allowing you to update text, change styles, show/hide content, and respond to user actions in real time.*